

Scalar & Binary Quantization, Worked Out by Hand

The two cheapest ways to shrink an embedding — map each `float32` to an `int8` ($4\times$ smaller), or to a single sign bit ($32\times$ smaller, distance by XOR) — with the quantize/dequantize arithmetic and reconstruction error computed by hand.

What this document does. It takes a small vector, scalar-quantizes it to 8-bit integers using a min-max scale and measures the round-trip error, then binary-quantizes a vector to one bit per dimension and computes a Hamming distance. It states the exact compression ratios and when each is safe. Numbers from Appendix A.

Companion: product quantization (#10) is the higher-fidelity option; recall@k (#12) measures the accuracy each one trades away.

1 Scalar quantization: `float32` $\rightarrow$ `int8`

A `float32` uses 32 bits to store one number to absurd precision. Embedding components rarely need that. Scalar quantization linearly maps the value range $[\min, \max]$ onto the 256 integer levels $0 \dots 255$:

$$q_i = \text{round}\left(\frac{v_i - \min}{\max - \min} \times 255\right), \quad \tilde{v}_i = \min + \frac{q_i}{255}(\max - \min)$$

The first equation quantizes (store q_i as one byte); the second dequantizes for computation. Memory drops from 32 to 8 bits per component — exactly $4\times$.

Worked example. $\mathbf{v} = (-0.8, 0.2, 1.0, -0.4, 0.6)$, so $\min = -0.8$, $\max = 1.0$, $\text{range} = 1.8$. For the second component $v_2 = 0.2$:

$$q_2 = \text{round}\left(\frac{0.2 - (-0.8)}{1.8} \times 255\right) = \text{round}(141.67) = 142.$$

The full code and its dequantization:

v_i	-0.8	0.2	1.0	-0.4	0.6
q_i (<code>uint8</code>)	0	142	255	57	198
$\tilde{v}_i$	-0.8	0.2024	1.0	-0.3976	0.5976
error	0	+0.0024	0	+0.0024	-0.0024

The endpoints are exact; interior values carry an error bounded by half a step, $\frac{1}{2} \cdot \frac{1.8}{255} = 0.0035$. Tiny — and embeddings are robust to it.

Intuition

Scalar quantization is a ruler with 256 tick marks laid across your value range. Any real value snaps to the nearest tick. The reconstruction error is at most half a tick, so the only way to get hurt is if a few rare outliers stretch $[\min, \max]$ so wide that the ticks become

coarse for everyone else. That is why per-vector or per-dimension ranges (and outlier clipping) matter in practice.

2 Binary quantization: one bit per dimension

Push compression to the limit: keep only the *sign* of each component.

$$b_i = \begin{cases} 1 & v_i > 0 \\ 0 & v_i \leq 0 \end{cases}$$

A D -dim `float32` vector ($32D$ bits) becomes D bits — a $32\times$ reduction. The magic is the distance: similarity between two binary codes is the *Hamming distance* (number of differing bits), computed as a hardware XOR plus a popcount — a handful of CPU cycles for hundreds of dimensions.

$$d_H(\mathbf{a}, \mathbf{b}) = \text{popcount}(\mathbf{b}_a \oplus \mathbf{b}_b).$$

Worked example. Sign-quantize two 8-dim vectors:

a	0.3	-0.1	0.8	-0.5	0.2	-0.9	0.4	-0.2
b_a	1	0	1	0	1	0	1	0
b	0.1	0.2	-0.3	-0.6	0.5	-0.4	-0.1	0.7
b_b	1	1	0	0	1	0	0	1
⊕	0	1	1	0	0	0	1	1

$d_H = 0+1+1+0+0+0+1+1 = 4$ differing bits out of 8. Smaller Hamming distance = more agreement in sign pattern = more similar direction.

Mini-example · the compression ladder

The same 768-dim embedding, three ways:

scheme	bits/vector	vs float32
float32	$768 \times 32 = 24576$	$1\times$
scalar int8	$768 \times 8 = 6144$	$4\times$ smaller
binary	$768 \times 1 = 768$	$32\times$ smaller

Each rung down trades fidelity for footprint and speed. Binary is fast enough to scan *billions* of vectors as a coarse filter, then re-rank the survivors with a higher-fidelity metric.

3 When to use which

- **Scalar (int8):** the safe default. $4\times$ smaller, sub-1% recall loss on most models, trivial to implement. Reach for it first.
- **Binary:** extreme scale or memory pressure. $32\times$ smaller and blazingly fast (XOR+popcount), but lossy — best as a *first-stage filter* feeding an exact or scalar re-rank, not as the final ranker.
- **Product quantization (#10):** when you need more fidelity than binary at better compression than scalar — the middle path, at the cost of codebook training.

Whichever you pick, validate the recall you actually keep (#12) before shipping — compression that quietly halves recall is a bug, not a feature.

Equation cheat-sheet

$$q_i = \text{round}\left(\frac{v_i - \min}{\max - \min} 255\right), \quad \tilde{v}_i = \min + \frac{q_i}{255}(\max - \min) \quad (4\times, \text{err} \leq \frac{\text{range}}{510})$$

$$b_i = [v_i > 0], \quad d_H = \text{popcount}(\mathbf{b}_a \oplus \mathbf{b}_b) \quad (32\times)$$

Appendix A · Reference implementation

```
import numpy as np
```

```
def scalar_q(v):
    v = np.asarray(v, float); lo, hi = v.min(), v.max()
    q = np.round((v - lo) / (hi - lo) * 255).astype(np.uint8)
    deq = lo + q / 255 * (hi - lo)
    return q, deq
```

```
scalar_q([-0.8,0.2,1.0,-0.4,0.6])[0]    # [0 142 255 57 198]
```

```
def hamming(a, b):
    ba, bb = (np.array(a) > 0), (np.array(b) > 0)
    return int((ba != bb).sum())
```

```
hamming([0.3,-0.1,0.8,-0.5,0.2,-0.9,0.4,-0.2],
        [0.1,0.2,-0.3,-0.6,0.5,-0.4,-0.1,0.7])    # 4
```

Internal learning material. Numbers reproducible from the appendix code. v1.0